ResumeLens

Formal Language-Based Resume Screening

1 · Detection
Regular expressions

→

2 · Transformation
Finite-state transducers

→

3 · Classification
Finite automata

→

4 · Validation
Context-free grammar

Juan Restrepo · Samuel Granda · Daniel Varela

CONTEXT

Problem and Relevance

The same qualification is written many ways

JS, Javascript → **JAVASCRIPT**

NodeJS, Node.js → **NODE_JS**

sklearn, scikit learn, Scikit-learn → **SCIKIT_LEARN**

Each profile needs a particular combination

Full Stack = web language + frontend + backend + database + version control

RELEVANCE

Same résumé, same verdict: a fully formal pipeline with no LLM, so nothing is guessed.

OUT OF SCOPE

Does not rank candidates

Does not make hiring decisions

Uses no NLP and no machine learning

METHODOLOGY

Methodology and Architecture



Stage	Formal model	Library	Input	Output
1 · **Detection**	Regular expressions	re	Résumé text	Fields + raw_skills
2 · **Transformation**	Finite-state transducers	pyformlang.fst	raw_skills	Canonical names, profile order
3 · **Classification**	Deterministic finite automata	pyformlang .finite_automaton	Canonical sequence	ACCEPTED / REJECTED + reason, per profile
4 · **Validation**	Context-free grammar	textX	Candidate report document	textX model → HTML5 / Markdown, or syntax error

STAGE 1 · DETECTION

Regular Expressions

Regular languages: 22 named compiled patterns + 1 separator

Alphabet · Unicode text, line endings normalized

Boundaries & lookarounds · Git ∉ Digit, js ∉ Node.js, C++ / C#

Multiline anchors ^ $ · a header is a whole line

Named groups · a stable interface for Stage 2

(?P<skill>) (?P<user>) (?P<domain>) (?P<years>) (?P<role>) (?P<company>) (?P<degree>)

Kleene: regex ⇔ NFA ⇔ DFA. Python re matches leftmost-first, not as a DFA.

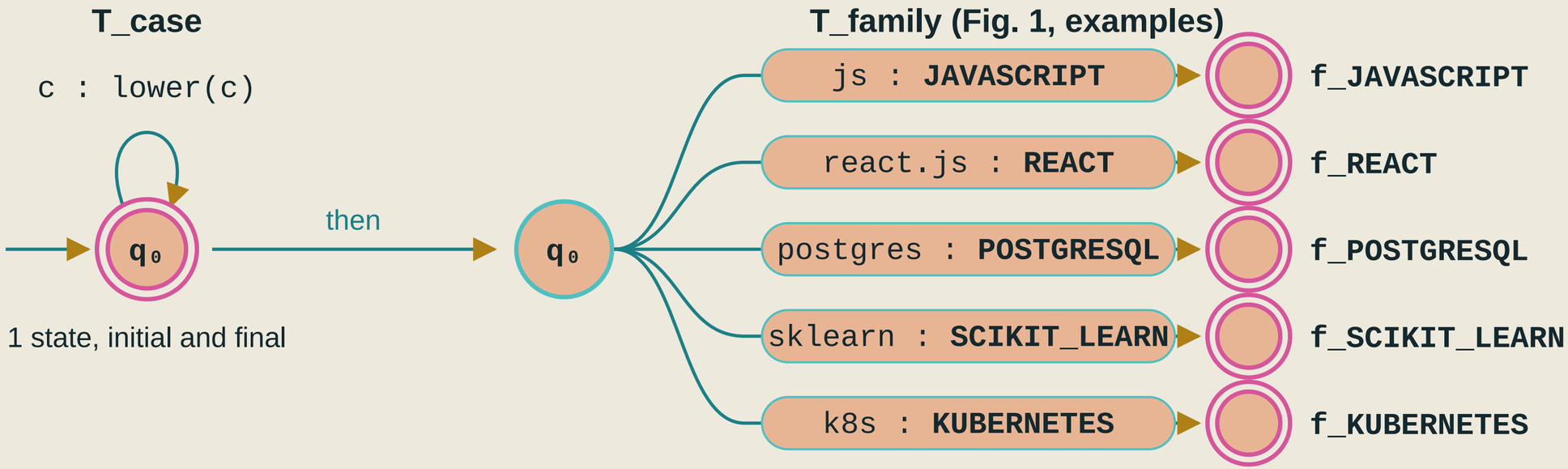
Detection · Wednesday Addams résumé

name	WEDNESDAY ADDAMS
e-mail	1
phone	1
links	2
education	2 entries
experience	2 entries
raw_skills	**[JS, React.js, NodeJS, Postgres, Git]**

STAGE 2 · TRANSFORMATION

Finite-State Transducers

$$M = (Q, \Sigma, \Gamma, \delta, \omega, q_0, F)$$



Whole-word symbols · 7 families, 2–12 states each, deterministic

Unknown tokens and duplicates are reported, never guessed

Git, NodeJS, JS, Postgres, React.js

⇒ JAVASCRIPT, REACT, NODE_JS, POSTGRESQL, GIT

Full Stack order: language → frontend → backend → database → API → VCS

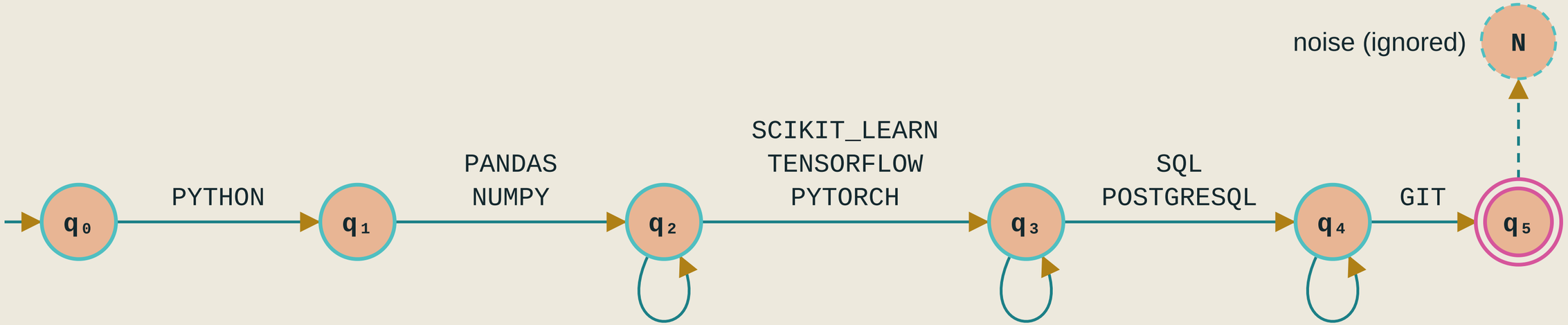
STAGE 3 · CLASSIFICATION

Finite Automata

$$M = (Q, \Sigma, \delta, q_0, F)$$

Σ = 50 canonical names · one transition per (state, symbol), no ε-moves

Fig. 2 · Machine Learning Engineer



self-loops: more skills of the same stage

L(P) = one skill from every required stage, in order

Optional stages may be skipped; known extra skills only after the profile part (N)

PYTHON PANDAS TENSORFLOW POSTGRESQL GIT

ACCEPTED · reaches the final state q_5

STAGE 3 · PROFILES

Four Profiles, One Construction

Profile	Stages ([ ] = optional)	States
Full Stack Developer given by the course	web language → frontend framework → backend → database → [API] → version control	8
Machine Learning Engineer given by the course	base language → data library → ML framework → [ML practice] → database → version control	8
DevOps Engineer team proposal	language → container → orchestration → infrastructure as code → [CI/CD] → [cloud] → version control	9
Data Engineer team proposal	language → data processing → workflow → database → [cloud] → version control	8

A profile is data: one build_automaton builds all four

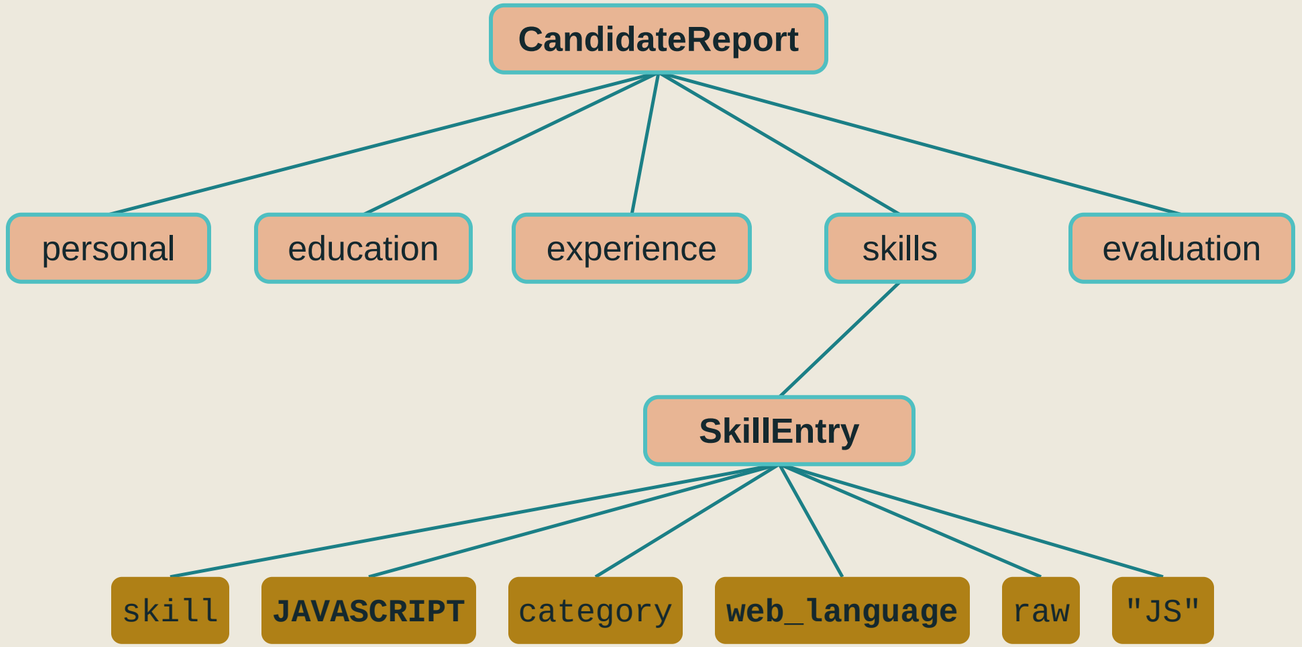
A candidate may be accepted by several profiles

STAGE 4 · VALIDATION

Context-Free Grammar

$$G = (V, \Sigma, S, P)$$

```
CandidateReport:
'candidate' '{'
personal=PersonalInfo
education=EducationSection
experience=ExperienceSection
skills=SkillSection
evaluation=EvaluationSection
'}' ;
SkillEntry: 'skill' canonical
'category' category 'raw' string ;
Comment: /\/\/.*$/ ;
```



terminals: keywords, punctuation, inline-regex tokens

5 mandatory blocks, fixed order · skills ≥ 1

Exactly one verdict rule per profile

Valid → textX model → HTML5 page or Markdown · malformed → error with position

DESIGN

Design Decisions

Stage 1 · T_case
Unicode characters

→

T_family
Whole-word symbols

→

Stage 3 · DFA
50 canonical names

Decision	Choice	Rationale
Abstraction level	Only explicitly written qualifications	Nothing is inferred or guessed
Grammar structure	5 mandatory blocks, fixed order	An incomplete report is a syntax error
Classification	One skill per required stage, in order	An ordered regular language: a DFA suffices
Canonical order	Sort by profile stage	Verdict independent of writing order
Noise handling	Extra known skills → state N; unknown → report	Nothing is lost or silently dropped

RESULTS

Results and Interpretation

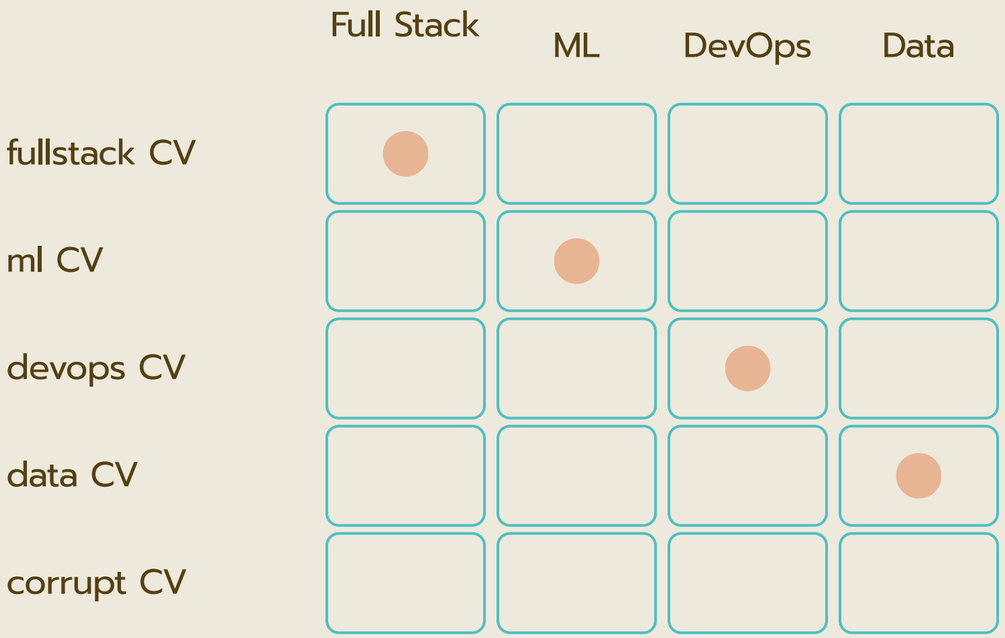


Fig. 3 · Verdict matrix · pink = ACCEPTED, empty = REJECTED

5/5

résumés with the expected verdict

1699

pytest tests pass, incl. every automaton vs. an independent regex

CLASSIFICATION · Full Stack résumé, ML profile

```
REJECTED · POSTGRESQL (position 1) belongs to stage 'database' but the required stage(s) base_language, data_library, ml_framework must come before it
```

VALIDATION · corrupt résumé (no skills)

```
DFA: the sequence is empty; every required stage needs at least one skill
textX: line 10, column 5: Expected 'skill'
```

DISCUSSION

Limitations and Improvements

LIMITATIONS

Regex	Names: 2–4 capitalized words; skills only from “Technical Skills”; 2 patterns backtrack quadratically (worst case)
FST	Only listed spellings; unknown ones reported, not guessed
DFA	Fixed sequential profiles; DevOps/Data are our proposal; ML database stage accepts any database
CFG	Grammar fixes exactly four profiles
Data	Five synthetic résumés only

IMPROVEMENTS

Unicode- and locale-aware patterns

User-definable profiles, grammar derived from them

Another model for profiles whose stages may appear in any order

Validation on real résumés

Formal models make every decision traceable: each verdict is a path through an automaton.

References · Hopcroft, Motwani & Ullman, Introduction to Automata Theory, Languages, and Computation · Documentation of Python re, pyformlang and textX